

Отчет по практической 4

Разработка приложения по анализу погоды онлайн бесплатно без смс и регистрации

Для начала напишем код:

```
import flet as ft
import requests
from datetime import datetime
```

```
class WeatherData:
```

```
    def __init__(self, temperature, apparent_temperature, humidity,
precipitation, pressure, weather_code, wind_speed, wind_direction):
        self.temperature = temperature
        self.apparent_temperature = apparent_temperature
        self.humidity = humidity
        self.precipitation = precipitation
        self.pressure = pressure
        self.weather_code = weather_code
        self.wind_speed = wind_speed
        self.wind_direction = wind_direction
```

```
cities = {
    "Макеевка": (47.96, 37.96),
    "Донецк": (48.00, 37.80),
    "Ростов": (47.23, 39.72),
    "Москва": (55.75, 37.62),
    "Санкт-Петербург": (59.93, 30.31)
}
```

```
def get_weather_description(code):
```

```
descriptions = {
    0: "ясно", 1: "преимущественно ясно", 2: "переменная облачность", 3:
"пасмурно",
    45: "туман", 48: "изморозь", 51: "мелкий дождь", 53: "умеренный
дождь", 55: "сильный дождь",
    56: "ледяной дождь", 57: "сильный ледяной дождь", 61: "небольшой
дождь", 63: "дождь", 65: "сильный дождь",
    66: "ледяной дождь", 67: "сильный ледяной дождь", 71: "небольшой
снег", 73: "снег", 75: "сильный снег",
    77: "град", 80: "небольшой ливень", 81: "ливень", 82: "сильный
ливень", 85: "небольшой снегопад",
    86: "снегопад", 95: "гроза", 96: "гроза с градом", 99: "сильная гроза с
градом"
}
return descriptions.get(code, "неизвестно")
```

```
def evaluate_comfort(temp):
    if temp < -10: return "очень холодно"
    elif temp < 0: return "холодно"
    elif temp < 15: return "прохладно"
    elif temp < 25: return "комфортно"
    elif temp < 35: return "тепло"
    else: return "жарко"
```

```
def evaluate_wind(speed):
    if speed < 5: return "штиль"
    elif speed < 15: return "слабый ветер"
    elif speed < 25: return "умеренный ветер"
    else: return "сильный ветер"
```

```

def get_weather_data(lat, lon):
    url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=temperature_2m,apparent_temperature,relative_humidity_2m,precipitation,pressure_msl,windspeed_10m,winddirection_10m,weathercode&timezone=auto"
    response = requests.get(url)
    response.raise_for_status()
    data = response.json()
    current = data["current"]
    return WeatherData(
        temperature=current["temperature_2m"],
        apparent_temperature=current["apparent_temperature"],
        humidity=current["relative_humidity_2m"],
        precipitation=current["precipitation"],
        pressure=current["pressure_msl"],
        weather_code=current["weathercode"],
        wind_speed=current["windspeed_10m"],
        wind_direction=current["winddirection_10m"]
    )

```

```

async def main(page: ft.Page):
    page.title = "Экспертная система погоды"
    city_dropdown = ft.Dropdown(label="Выберите город",
options=[ft.dropdown.Option(city) for city in cities])
    update_button = ft.ElevatedButton("Обновить погоду")
    weather_blocks = ft.Column()

    def create_block(label, value, color):
        return ft.Container(
            content=ft.Column([

```

```
        ft.Text(label, size=14, weight=ft.FontWeight.BOLD),
        ft.Text(value, size=18)
    ], alignment=ft.MainAxisAlignment.CENTER),
    bgcolor=color,
    padding=10,
    border_radius=10,
    width=150,
    height=100
)
```

```
async def update_weather(e):
```

```
    if not city_dropdown.value:
```

```
        weather_blocks.controls.clear()
```

```
        weather_blocks.controls.append(ft.Text("Выберите город"))
```

```
        page.update()
```

```
        return
```

```
    try:
```

```
        lat, lon = cities[city_dropdown.value]
```

```
        data = get_weather_data(lat, lon)
```

```
        desc = get_weather_description(data.weather_code)
```

```
        comfort = evaluate_comfort(data.apparent_temperature)
```

```
        wind_eval = evaluate_wind(data.wind_speed)
```

```
        time_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
        weather_blocks.controls.clear()
```

```
        weather_blocks.controls.extend([
```

```
            ft.Text(f"Город: {city_dropdown.value} | Обновлено: {time_str}",
```

```
            size=16, weight=ft.FontWeight.BOLD),
```

```
            ft.Row([
```

```
                create_block("Температура", f"{data.temperature}°C",
```

```
                "maroon"),
```

```

        create_block("Ощущается как", comfort, "blue"),
        create_block("Влажность", f"{data.humidity}%", "green"),
        create_block("Ветер", f"{data.wind_speed} км/ч ({wind_eval})",
"orange"),

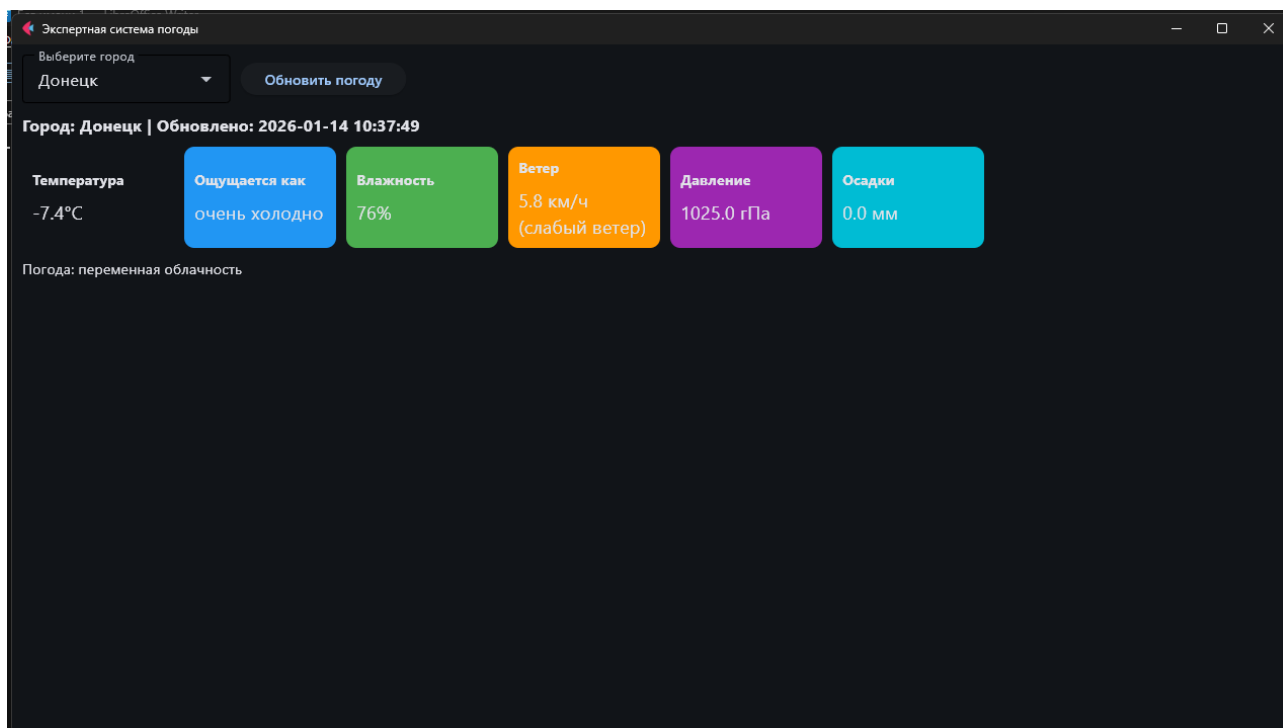
        create_block("Давление", f"{data.pressure} гПа", "purple"),
        create_block("Осадки", f"{data.precipitation} мм", "cyan")
    ], alignment=ft.MainAxisAlignment.START, wrap=True),
    ft.Text(f"Погода: {desc}", size=14)
])
except Exception as ex:
    weather_blocks.controls.clear()
    weather_blocks.controls.append(ft.Text(f"Ошибка: {str(ex)}"))
page.update()

update_button.on_click = update_weather
page.add(ft.Row([city_dropdown, update_button]), weather_blocks)

ft.app(target=main)

```

Потом посмотри интерфейс:



Ответы на вопрос:

1. Реляционная машина - система логического вывода, основанная на работе с отношениями (таблицами) и реляционными операциями (проекция, соединение и т.д.).
2. Преимущества у реляционного подхода в экспертных системах: простота, наглядность, использование мощного аппарата реляционной алгебры, эффективность хранения и обработки структурированных фактов.
3. Расширить систему для поддержки большего числа городов можно добавив новые факты (кортежи) в таблицу городов и правила, которые с ними работают. Масштабируемость заложена в структуре базы знаний.
4. Никакие погодные параметры не требуются в системе.
5. Реализовать автоматическое обновление данных из внешнего источника можно поставив цикл выполняющийся каждую секунду который отправляет запрос и по ответу обновляется интерфейс с этих данных.